

Épreuve de Programmation Orientée Objet 2

Licence Informatique — Sujet d'entraînement B

Streams et Command/MVC

Juin 2026

Durée : 2 heures. **Documents autorisés :** documentation de l'API Stream (`java.util.stream`) et des interfaces `Comparator` et `Collectors`.

Ce sujet est un sujet d'entraînement généré à partir du contenu du cours de POO2 (Iterator, Singleton/Enum, Strategy, Decorator, Streams, Command/MVC, Builder). Il n'a pas valeur de sujet officiel mais reprend le format et l'esprit des épreuves de M. Champesme.

Ce qui est évalué dans cette épreuve

Dans cette épreuve sont évaluées vos capacités à : - utiliser correctement le langage JAVA pour implémenter une classe ; - utiliser correctement l'API Stream pour exprimer des traitements sur des collections sans boucle explicite ; - effectuer des choix corrects pour la visibilité des champs et des méthodes ; - donner l'implémentation d'une classe en conformité avec son interface et son contrat ; - respecter la séparation des responsabilités de l'architecture MVC ; - mettre en œuvre les design patterns **Command** et l'architecture **MVC**.

1 API Stream

Exercice 1 On dispose d'une classe `Etudiant` possédant les méthodes `getNom()` (renvoie un `String`), `getNote()` (renvoie un `double`) et `getGroupe()` (renvoie un `String`). On dispose d'une `List<Etudiant>` `etudiants`.

En utilisant exclusivement l'API Stream (**sans aucune boucle `for` / `while` explicite**), donnez le code source complet des méthodes suivantes :

- `public static List<String> nomsAdmis(List<Etudiant> etudiants)` qui renvoie la liste des noms des étudiants ayant une note supérieure ou égale à 10, triés par ordre alphabétique.
- `public static double moyenneGroupe(List<Etudiant> etudiants, String groupe)` qui renvoie la moyenne des notes des étudiants appartenant au groupe donné (on renverra 0 si aucun étudiant du groupe n'est trouvé).
- `public static Map<String, List<Etudiant>> parGroupe(List<Etudiant> etudiants)` qui renvoie une association entre chaque nom de groupe et la liste des étudiants appartenant à ce groupe.
- `public static Optional<Etudiant> meilleurEtudiant(List<Etudiant> etudiants)` qui renvoie l'étudiant ayant la meilleure note (un `Optional` vide si la liste est vide).

IMPORTANT : toute solution utilisant une boucle `for` / `while` explicite plutôt que l'API Stream ne sera pas acceptée, même si le résultat produit est correct.

2 Design pattern *Command* et architecture *MVC*

Exercice 2 On souhaite implémenter, selon l'architecture **MVC**, un système de commandes pour un jeu d'aventure textuelle inspiré de Zork. Le **Modèle** est une classe `Salle` qui possède des méthodes `ajouterObjet(ObjetZork o)` et `retirerObjet(ObjetZork o)`, **sans aucun affichage à l'écran**. La **Vue** est une classe `Vue` qui possède une unique méthode `afficherMessage(String message)` (qui se contente d'appeler `System.out.println`). On souhaite que

chaque action du joueur soit représentée par un objet, selon le design pattern *Command*.

1. Donnez le code source complet d'une interface `Commande` adaptée à ce contexte, possédant une unique méthode abstraite permettant d'exécuter l'action représentée par la commande.
2. Donnez le code source complet d'une classe `CommandePoserObjet` qui implémente `Commande`. Son exécution doit : ajouter l'objet donné à la salle donnée (toutes deux fournies au constructeur), puis demander à la vue (également fournie au constructeur) d'afficher un message de la forme "Vous avez pose <nom de l'objet> dans la salle."
3. Donnez le code source complet d'une classe `Controleur` qui maintient un historique des commandes exécutées (`List<Commande>`) et qui possède une méthode `executer(Commande commande)` exécutant la commande puis l'ajoutant à l'historique.
4. En deux ou trois lignes, expliquez pourquoi le Modèle (la classe `Salle`) ne doit jamais appeler directement une méthode de la classe `Vue`, et quel principe de l'architecture MVC est ainsi respecté.

IMPORTANT : pour la classe `Salle`, en dehors de la définition des attributs, seules les méthodes `ajouterObjet` et `retirerObjet` sont demandées. Pour les classes de cet exercice, les commentaires et assertions ne sont pas demandés.

Sujet d'entraînement généré par Claude — Anthropic.